1. Introduction. Knuth's Stanford GraphBase page carries a list of Challenge Problems—"several yet-unsolved problems on which I'm hoping readers will make significant progress"—and calls this one of the most fun of them: *What is the biggest score Stanford can rack up over Harvard by a simple chain of results from that year?* The year is 1990, and the graph produced by GB_GAMES records its 638 college football games; if team u beat team v by d points we may chain the two together and claim that u is d points better than v, provided that no team appears twice in the chain.

The page keeps score. Knuth's own best was 2279, the chain that FOOTBALL describes; Buel Chandler improved it to 2448 by applying genetic programming ideas to Knuth's algorithms; and on 25 November 2001 Mark Cooke reported a chain worth 2473, together with a matching upper bound found in February 2002. So 2473 is optimal. The page records two more values: 2358 for Harvard over Stanford, and 2542 for Penn State over Columbia, which is the maximum over all pairs of teams.

This program proves all three of those numbers from scratch, and exhibits the optimal chains, in a fraction of a second. It is a companion to a longer program of mine that attacks the same challenge with iterated local search; that one climbs to within five points of the optimum but can never say that it has arrived. Here I do nothing but bound, and the bound turns out to be so sharp that the search hardly needs to search at all. That, rather than the numbers themselves, is what I found worth reporting.

2. The idea is an old one from the travelling salesman literature, applied to a longest-path problem. Drop the requirement that the chain be connected and keep only the degree constraints, and what is left is an assignment problem, which the Hungarian algorithm solves exactly in $O(n^3)$ steps. Every chain satisfies the degree constraints, so the assignment value is an upper bound. If the assignment happens to be a single path, that path is a chain achieving the bound and we are done; otherwise it contains a cycle, which no chain may contain, and we branch on which edge of that cycle to leave out.

Usage is

```
chain_bound [-from=Team] [-to=Team] [-c] [-t=limit] [-D=directory]
```

where `-from` and `-to` default to `Stanford` and `Harvard` and must be spelled as in `games.dat`; `-c` prints the winning chain game by game, in the format of the FOOTBALL demo; and `-t` is a time limit after which the program reports the best bounds it has reached instead of a proof (the default, 0, means no limit).

3. Here is the whole program. It reads the season, solves the root relaxation, dissects it for the reader, and then runs the branch-and-bound search.

```
package main
import (
    "container/heap"
    "flag"
    "fmt"
    "log"
    "math"
    "time"
    "github.com/sjnam/go-sgb/gbgames"
)
⟨ Type declarations 5 ⟩
⟨ Subroutines 6 ⟩
func main( ) {
    ⟨ Read the command line 4 ⟩
    p, err := buildProblem(∗dir, ∗from, ∗to)
    if err ≠ Λ {
        log.Fatal(err)
    }
    ⟨ Solve the problem and report 32 ⟩
}
```

4. ⟨ Read the command line 4 ⟩ ≡

```
from := flag.String("from", "Stanford", "team␣the␣chain␣starts␣at")
to := flag.String("to", "Harvard", "team␣the␣chain␣ends␣at")
dir := flag.String("D", "/usr/local/sgb/data", "directory␣holding␣games.dat")
show := flag.Bool("c", false, "print␣the␣chain␣game␣by␣game")
limit := flag.Duration("t", 0, "give␣up␣after␣this␣long␣(0␣=␣never)")
flag.Parse( )
```

This code is used in section 3.

5. The problem. A chain is a simple path in the graph of GB_GAMES: a sequence of distinct teams in which consecutive teams played each other. If v follows u in the chain we earn $del[u][v]$, the number of points by which u beat v in their game (a negative number if u actually lost). So the problem is to find the simple path from *start* to *goal* that maximizes $\sum del$. This is a longest-path problem, and longest path is **NP**-hard; but there are only 120 teams here, and as we shall see the instance is far from the worst case.

⟨ Type declarations 5 ⟩ ≡

```
type problem struct {
    n int
    names []string
    nick []string   // nicknames, used only when printing the chain
    exists [][]bool
    del [][]int64
    uScore [][]int64   // points scored by the first team of the pair
    vScore [][]int64   // points scored by the second team
    date [][]int64   // day of the game
    start int
    goal int
}
```

This code is also defined in sections 11, 12, 21, 26.

This code is used in section 3.

6. Two teams occasionally met twice, once during the season and once in a bowl game. When we are maximizing we may always use the better of the two results, so *del* keeps the larger point spread and remembers that game's scores and date along with it.

⟨ Subroutines 6 ⟩ ≡

```
func buildProblem(dir, from, to string)(*problem, error) {
    g, err := gbgames.Games(0, 0, 0, 0, 0, 0, 0, 0, dir)
    if err ≠ Λ {
        return Λ, err
    }
    n := int(g.N)
    p := &problem{n: n, names: make([]string, n), nick: make([]string, n),
        start: −1, goal: −1}
    ⟨ Allocate the matrices 7 ⟩
    ⟨ Fill the matrices from the vertices and arcs 8 ⟩
    if p.start < 0 ∨ p.goal < 0 {
        return Λ, fmt.Errorf("unknown team: %q or %q", from, to)
    }
    return p, Λ
}
```

This code is also defined in sections 9, 14, 18, 20, 22, 27.

This code is used in section 3.

7. Only *del* needs a nonzero initial value; a missing game must never look attractive. The magnitude of a point spread is at most a few hundred, so -2^{31} is a comfortable stand-in for $-\infty$.

⟨ Allocate the matrices 7 ⟩ ≡

```
const negInf = int64(math.MinInt32)
p.exists = make([][]bool, n)
p.del = make([][]int64, n)
p.uScore = make([][]int64, n)
p.vScore = make([][]int64, n)
p.date = make([][]int64, n)
for i := 0; i < n; i++ {
    p.exists[i] = make([]bool, n)
    p.del[i] = make([]int64, n)
    p.uScore[i] = make([]int64, n)
    p.vScore[i] = make([]int64, n)
    p.date[i] = make([]int64, n)
    for j := range p.del[i] {
        p.del[i][j] = negInf
    }
}
```

This code is used in section 6.

8. In the arcs of a GB_GAMES graph, *a.Len* is the number of points scored by the team at the tail and *a.Partner.Len* the number scored by the team at the tip; *a.B.I* is the day of the game.

⟨ Fill the matrices from the vertices and arcs 8 ⟩ ≡

```
for i := 0; i < n; i++ {
    v := &g.Vertices[i]
    p.names[i] = v.Name
    p.nick[i] = v.Y.S
    if v.Name ≡ from {
        p.start = i
    }
    if v.Name ≡ to {
        p.goal = i
    }
    for a := range v.AllArcs() {
        j := int(g.Index(a.Tip))
        if d := a.Len − a.Partner.Len; d > p.del[i][j] {
            p.del[i][j] = d
            p.uScore[i][j] = a.Len
            p.vScore[i][j] = a.Partner.Len
            p.date[i][j] = a.B.I
        }
        p.exists[i][j] = true
    }
}
```

This code is used in section 6.

9. We will want to add up the value of a chain now and then, if only to check that the answer really is what we claim it is.

⟨ Subroutines 6 ⟩ +≡

```
func (p *problem) total(path []int) int64 {
    var t int64
    for k := 0; k + 1 < len(path); k++ {
        t += p.del[path[k]][path[k + 1]]
    }
    return t
}
```

10. The assignment relaxation. Look at a chain and count degrees. The first team has one outgoing edge and no incoming one; the last team has one incoming edge and none outgoing; every other team either passes the chain along—one edge in, one edge out—or takes no part at all.

Now keep those degree constraints and throw away the requirement that the whole thing hang together in one piece. What survives is exactly an assignment problem: choose for each team a successor, and for each team a predecessor, consistently. Its solutions are the sets consisting of one path from *start* to *goal* together with any number of vertex-disjoint cycles among the remaining teams. Every chain is such a solution, so the optimum of the relaxation is an upper bound on the optimum chain.

The bound comes with a certificate attached. If the optimal assignment has no cycle of positive value, then its path alone already achieves the whole relaxation value—cycles of value zero contribute nothing—and upper bound meets lower bound. If some cycle $e_1, \ldots, e_m$ has positive value, no chain can contain all of it, so at least one e_k must be left out. We therefore split the problem into m subproblems: in the kth, edge e_k is forbidden and $e_1, \ldots, e_{k-1}$ are forced. Splitting on the index of the first omitted edge makes the subproblems disjoint, which is the classical subtour-elimination branching rule.

11. A node of the search tree records its forbidden and forced edges, the successor mapping of the relaxed solution under those restrictions, and the resulting bound. The **bounder** carries the problem, the base cost matrix, and the incumbent—the best chain found so far, which is our lower bound.

⟨ Type declarations 5 ⟩ +≡

```
type bbNode struct {
    bound int64
    banned [][2]int
    forced [][2]int
    succ []int
}
type bounder struct {
    p *problem
    rows []int   // row number to team (every team but goal)
    cols []int   // column number to team (every team but start)
    rowOf []int
    colOf []int
    base [][]int64
    inc int64   // the incumbent's value
    incPath []int
}
```

12. A forbidden entry gets a cost so large that any assignment using one is recognizable by its total. Costs are at most a few hundred and the matrix has 120 rows, so 2^{40} leaves room to spare.

⟨ Type declarations 5 ⟩ +≡

```
const forbid = int64(1) << 40
```

13. Rows are indexed by the teams that must choose a successor, that is, all teams but *goal*; columns by the teams that must choose a predecessor, all teams but *start*. The diagonal entry for a middle team is the "take no part" option and costs nothing. Note that *start* has no column and *goal* has no row, so neither of them has a diagonal entry available: the first team is forced to choose a real successor and the last a real predecessor, which is just what we want. Since the Hungarian algorithm minimizes, we negate.

⟨ Set up the rows, the columns, and the base matrix 13 ⟩ ≡

```
bd.rowOf = make([]int, p.n)
bd.colOf = make([]int, p.n)
for v := 0; v < p.n; v++ {
    bd.rowOf[v], bd.colOf[v] = -1, -1
    if v ≠ p.goal {
        bd.rowOf[v] = len(bd.rows)
        bd.rows = append(bd.rows, v)
    }
    if v ≠ p.start {
        bd.colOf[v] = len(bd.cols)
        bd.cols = append(bd.cols, v)
    }
}
bd.base = make([][]int64, len(bd.rows))
for r, uu := range bd.rows {
    bd.base[r] = make([]int64, len(bd.cols))
    for c, vv := range bd.cols {
        switch {
        case uu ≡ vv:
            bd.base[r][c] = 0   // take no part
        case p.exists[uu][vv]:
            bd.base[r][c] = -p.del[uu][vv]
        default:
            bd.base[r][c] = forbid
        }
    }
}
```

This code is used in section 22.

14. The Hungarian algorithm. Here is the $O(n^3)$ form that maintains dual potentials u and v and grows one augmenting path per row. (ASSIGN_LISA contains a Hungarian method too, but that one is a standalone program tuned for rectangular matrices, so a short square-matrix version of my own is more convenient here.) On return, $match[j]$ is the row assigned to column j, numbered from 1; the results reported are the column chosen by each row and the total cost.

⟨ Subroutines 6 ⟩ +≡

```
func hungarian(a [][]int64)([]int, int64) {
    n := len(a)
    const inf = int64(math.MaxInt64/8)
    u := make([]int64, n+1)
    v := make([]int64, n+1)
    match := make([]int, n+1)
    way := make([]int, n+1)   // previous column on the augmenting path
    minv := make([]int64, n+1)
    used := make([]bool, n+1)
    for i := 1; i ≤ n; i++ {
        ⟨ Find an augmenting path from row i and update the assignment 15 ⟩
    }
    res := make([]int, n)
    total := int64(0)
    for j := 1; j ≤ n; j++ {
        res[match[j]-1] = j-1
        total += a[match[j]-1][j-1]
    }
    return res, total
}
```

15. Column 0 is a fictitious column that holds the row we are currently trying to place. We walk to the unused column of least reduced cost until we reach a column that nobody occupies, then walk back along *way* reassigning as we go.

⟨ Find an augmenting path from row i and update the assignment 15 ⟩ ≡

```
match[0] = i
j0 := 0
for j := 0; j ≤ n; j++ {
    minv[j] = inf
    used[j] = false
}
for {
    ⟨ Step to the unused column of least reduced cost 16 ⟩
    if match[j0] ≡ 0 {
        break
    }
}
⟨ Retrace way, reassigning columns 17 ⟩
```

This code is used in section 14.

16. From the row *i0* that occupies the current column *j0* we refresh the reduced cost of every unused column, find the smallest, and shift the potentials by that amount so that it becomes tight.

⟨ Step to the unused column of least reduced cost 16 ⟩ ≡

```
used[j0] = true
i0, j1, delta := match[j0], 0, inf
for j := 1; j ≤ n; j++ {
    if used[j] {
        continue
    }
    if cur := a[i0 − 1][j − 1] − u[i0] − v[j]; cur < minv[j] {
        minv[j] = cur
        way[j] = j0
    }
    if minv[j] < delta {
        delta = minv[j]
        j1 = j
    }
}
for j := 0; j ≤ n; j++ {
    if used[j] {
        u[match[j]] += delta
        v[j] −= delta
    } else {
        minv[j] −= delta
    }
}
j0 = j1
```

This code is used in section 15.

17. ⟨ Retrace *way*, reassigning columns 17 ⟩ ≡

```
for j0 ≠ 0 {
    j1 := way[j0]
    match[j0] = match[j1]
    j0 = j1
}
```

This code is used in section 15.

18. Solving one node. To evaluate a node we copy the base matrix, impose its restrictions, and call *hungarian*. Forbidding an edge is a single entry; forcing one means forbidding every other entry in its row and in its column. If the answer still uses a forbidden entry—which shows up as a total of *forbid* or more—the node is infeasible and we drop it.

```
⟨ Subroutines 6 ⟩ +≡
  func (bd *bounder) solve(banned, forced [][2]int) *bbNode {
      m := make([][]int64, len(bd.base))
      for r := range m {
          m[r] = append([]int64(Λ), bd.base[r]...)
      }
      for _, e := range banned {
          m[bd.rowOf[e[0]]][bd.colOf[e[1]]] = forbid
      }
      for _, e := range forced {
          ⟨ Forbid everything that competes with the forced edge e 19 ⟩
      }
      res, total := hungarian(m)
      if total ≥ forbid {
          return Λ
      }
      succ := make([]int, bd.p.n)
      for r, uu := range bd.rows {
          succ[uu] = bd.cols[res[r]]
      }
      return &bbNode{bound: −total, banned: banned, forced: forced, succ: succ}
  }
```

19. ⟨ Forbid everything that competes with the forced edge *e* 19 ⟩ ≡

```
  r, c := bd.rowOf[e[0]], bd.colOf[e[1]]
  for cc := range m[r] {
      if cc ≠ c {
          m[r][cc] = forbid
      }
  }
  for rr := range m {
      if rr ≠ r {
          m[rr][c] = forbid
      }
  }
```

This code is used in section 18.

20. To decide whether a node is finished we look for a cycle of positive value in its relaxed solution. Following successors from *start* marks the path; a team that points at itself is sitting the game out; whatever is left forms the cycles. Among the positive ones we return the shortest, since the number of children is the length of the cycle we branch on.

⟨ Subroutines 6 ⟩ +≡

```
func (bd *bounder) posCycle(succ []int) []int {
    p := bd.p
    seen := make([]bool, p.n)
    for v := p.start; v ≠ p.goal; v = succ[v] {
        seen[v] = true
    }
    seen[p.goal] = true
    var best []int
    for s := 0; s < p.n; s++ {
        if seen[s] ∨ succ[s] ≡ s {
            continue
        }
        var cyc []int
        tot := int64(0)
        for v := s; ¬seen[v]; v = succ[v] {
            seen[v] = true
            cyc = append(cyc, v)
            tot += p.del[v][succ[v]]
        }
        if tot > 0 ∧ (best ≡ Λ ∨ len(cyc) < len(best)) {
            best = cyc
        }
    }
    return best
}
```

21. Best-first search. Nodes wait in a heap ordered by decreasing bound, so the top of the heap is always a valid upper bound for the whole problem. That has a pleasant consequence: if we run out of time we can still report honestly how far the gap has been narrowed, instead of having nothing to show.

⟨ Type declarations 5 ⟩ +≡

```
type bbHeap []*bbNode
func (h bbHeap) Len() int { return len(h) }
func (h bbHeap) Less(i, j int) bool { return h[i].bound > h[j].bound }
func (h bbHeap) Swap(i, j int) { h[i], h[j] = h[j], h[i] }
func (h *bbHeap) Push(x any) {*h = append(*h, x.(*bbNode)) }
func (h *bbHeap) Pop() any {
    old := *h
    n := len(old)
    x := old[n - 1]
    *h = old[:n - 1]
    return x
}
```

22. The search needs no heuristic to prime it; it discovers its own incumbent along the way, because a node whose relaxation has no positive cycle hands us a genuine chain. We start the incumbent below every attainable value.

⟨ Subroutines 6 ⟩ +≡

```
func solveExactly(p *problem, limit time.Duration)(int64, int64, []int, int, bool) {
    bd := &bounder{p: p, inc: math.MinInt32}
    ⟨ Set up the rows, the columns, and the base matrix 13 ⟩
    root := bd.solve(Λ, Λ)
    if root ≡ Λ {
        log.Fatal("no chain exists at all")
    }
    fmt.Printf("relaxation bound %d\n", root.bound)
    ⟨ Dissect the root relaxation 30 ⟩
    h := &bbHeap{root}
    heap.Init(h)
    nodes, proven := 0, false
    deadline := time.Now().Add(limit)
    for h.Len() > 0 {
        if limit > 0 ∧ time.Now().After(deadline) {
            break
        }
        nd := heap.Pop(h).(*bbNode)
        if nd.bound ≤ bd.inc {
            proven = true
            break
        }
        nodes++
        ⟨ Accept nd as a chain, or branch on one of its cycles 23 ⟩
    }
    ⟨ Determine the final bounds 25 ⟩
    return ub, bd.inc, bd.incPath, nodes, proven
}
```

23. If the popped node has no positive cycle its path is a real chain worth exactly its bound, and since we pop in decreasing order of bound it is the best chain we could still have hoped for from this node; we record it and stop expanding here. Otherwise we generate the children, keeping only those that are feasible and still promising.

⟨ Accept *nd* as a chain, or branch on one of its cycles 23 ⟩ ≡

```
cyc := bd.posCycle(nd.succ)
if cyc ≡ Λ {
    if nd.bound > bd.inc {
        bd.inc = nd.bound
        bd.incPath = bd.incPath[:0]
        for v := p.start; ; v = nd.succ[v] {
            bd.incPath = append(bd.incPath, v)
            if v ≡ p.goal {
                break
            }
        }
    }
    continue
}
for k := range cyc {
    ⟨ Make the child that omits the kth edge of cyc 24 ⟩
}
```

This code is used in section 22.

24. ⟨ Make the child that omits the *k*th edge of *cyc* 24 ⟩ ≡

```
banned := append(append([][2]int(Λ), nd.banned...),
    [2]int{cyc[k], cyc[(k + 1) % len(cyc)]})
forced := append([][2]int(Λ), nd.forced...)
for j := 0; j < k; j++ {
    forced = append(forced, [2]int{cyc[j], cyc[j + 1]})
}
if child := bd.solve(banned, forced); child ≠ Λ ∧ child.bound > bd.inc {
    heap.Push(h, child)
}
```

This code is used in section 23.

25. The search ends in one of three ways: the heap empties, a popped bound falls to the incumbent, or the clock runs out. In the first two cases the incumbent is optimal. In the third the best remaining bound in the heap is what we can still claim.

⟨ Determine the final bounds 25 ⟩ ≡

```
ub := bd.inc
if ¬proven ∧ h.Len( ) > 0 {
    if top := (*h)[0].bound; top > ub {
        ub = top
    }
} else {
    proven = true
}
```

This code is used in section 22.

26. Anatomy of the root relaxation. It is worth asking why the bound is as good as it is, and the answer is visible if we take the relaxed solution apart. Every assignment solution splits into one path, some disjoint cycles, and the teams that chose their diagonal entry; so let us weigh the pieces separately.

```
⟨ Type declarations 5 ⟩ +≡
    type piece struct {
        len int   // how many teams this piece passes through
        tot int64   // how many points it earns
        head int   // a team to name it by
    }
```

27.

```
⟨ Subroutines 6 ⟩ +≡
    func (bd *bounder) decompose(succ []int)(int, int64, []piece, int) {
        p := bd.p
        seen := make([]bool, p.n)
        plen, ptot := 0, int64(0)
        for v := p.start; v ≠ p.goal; v = succ[v] {
            seen[v] = true
            ptot += p.del[v][succ[v]]
            plen++
        }
        seen[p.goal] = true
        var cycles []piece
        skipped := 0
        for s := 0; s < p.n; s++ {
            ⟨ Collect the piece that s opens 28 ⟩
        }
        return plen, ptot, cycles, skipped
    }
```

28.

```
⟨ Collect the piece that s opens 28 ⟩ ≡
    if seen[s] {
        continue
    }
    if succ[s] ≡ s {
        seen[s] = true
        skipped++
        continue
    }
    c := piece{head: s}
    for v := s; ¬seen[v]; v = succ[v] {
        seen[v] = true
        c.len++
        c.tot += p.del[v][succ[v]]
    }
    cycles = append(cycles, c)
```

This code is used in section 27.

29. For Stanford to Harvard the dissection comes out like this:

piece	size	points
path (Stanford to Harvard)	43 games	1058
cycle 1 (Texas A&M …)	58 teams	+1156
cycle 2 (Temple …)	5 teams	+79
cycle 3 (Princeton …)	8 teams	+157
cycle 4 (Pennsylvania …)	3 teams	+48
teams sitting out	2 teams	—
total		2498

There, in one table, is everything the relaxation is allowed to get away with. A chain would have to thread all 120 teams onto a single string; the relaxation instead runs a short path of 43 games and lets the other teams form little rings of their own, harvesting good games with no obligation to connect them to anything. And yet all that freedom is worth only $2498 - 2473 = 25$ points, about one percent. That is why the search finishes in 351 nodes: the relaxation is standing almost on top of the answer. Had the gap been wide, the tree would have exploded and heuristics would still be all we had.

30. ⟨ Dissect the root relaxation 30 ⟩ ≡

```
plen, ptot, cycles, skipped := bd.decompose(root.succ)
var ctot int64
for _, c := range cycles {
    ctot += c.tot
}
fmt.Printf("␣␣path␣of␣%d␣games␣worth␣%d,␣%d␣cycles␣worth␣%+d,␣%d␣teams␣sitting␣out\n",
    plen, ptot, len(cycles), ctot, skipped)
for _, c := range cycles {
    fmt.Printf("␣␣␣␣cycle␣of␣%d␣teams,␣%+d␣␣[%s␣...]\n",
        c.len, c.tot, p.names[c.head])
}
```

This code is used in section 22.

31. Results. Running the program on the three instances mentioned in the introduction gives

instance	root bound	optimum	nodes
Stanford to Harvard	2498	2473	351
Harvard to Stanford	2367	2358	51
Penn State to Columbia	2542	2542	1

all three agreeing with the values Cooke reported. The last two take a few hundredths of a second and the first about three seconds, nearly all of it spent in the 351 Hungarian calls.

The third instance is the prettiest: its optimal assignment has no positive cycle at all. The very first Hungarian call returns a path through 117 games worth 2542 and no cycles, which is to say it hands us the chain and the proof in the same breath.

I find the last column the interesting one. The challenge has stood since 1993 and was answered by an eight-year campaign of increasingly clever heuristics; but the instance itself, approached from above, is nearly trivial. My own local-search program spent a hundred and fifty seconds to get within five points of the optimum and could not certify even that. So the moral is not that heuristics were the wrong tool, but that it is worth measuring the relaxation gap before assuming that **NP**-hardness describes your instance.

Here, finally, is what the first instance looks like when the program is asked to show its work, with all but the ends of the chain elided:

```
relaxation bound 2498
  path of 43 games worth 1058, 4 cycles worth +1440, 2 teams sitting out
    cycle of 58 teams, +1156  [Texas A\&M ...]
    cycle of 5 teams, +79  [Temple ...]
    cycle of 8 teams, +157  [Princeton ...]
    cycle of 3 teams, +48  [Pennsylvania ...]
Stanford to Harvard:  upper bound 2473, chain +2473 (351 nodes, proven optimal)
 Oct 06:  Stanford Cardinal 36, Notre Dame Fighting Irish 31 (+5)
 Oct 20:  Notre Dame Fighting Irish 29, Miami Hurricanes 20 (+14)
 Jan 01:  Miami Hurricanes 46, Texas Longhorns 3 (+57)
 ...
 Sep 29:  Bucknell Bisons 42, Cornell Big Red 21 (+2435)
 Nov 10:  Cornell Big Red 41, Columbia Lions 0 (+2476)
 Sep 15:  Columbia Lions 6, Harvard Crimson 9 (+2473)
```

The last line is the dip promised earlier: Columbia lost to Harvard, so the running total falls from 2476 to 2473 on the very step that completes the optimal chain.

32. ⟨ Solve the problem and report 32 ⟩ ≡

```
ub, lb, chain, nodes, proven := solveExactly(p, *limit)
verdict := "not␣proven"
if proven {
    verdict = "proven␣optimal"
}
fmt.Printf("%s␣to␣%s:␣upper␣bound␣%d,␣chain␣%+d␣(%d␣nodes,␣%s)\n",
    *from, *to, ub, lb, nodes, verdict)
⟨ Check that the chain is genuine 33 ⟩
if *show {
    ⟨ Print the chain game by game 34 ⟩
}
```

This code is used in section 3.

33. A proof is worth no more than the checking of it, so before printing we verify mechanically that the answer is a simple path, that each of its edges is a game that was really played, and that its value is what we said.

⟨ Check that the chain is genuine 33 ⟩ ≡

```
seen := make(map[int]bool)
for k, v := range chain {
    if seen[v] {
        log.Fatalf("team␣%s␣appears␣twice!", p.names[v])
    }
    seen[v] = true
    if k + 1 < len(chain) ∧ ¬p.exists[v][chain[k + 1]] {
        log.Fatalf("%s␣never␣played␣%s!", p.names[v], p.names[chain[k + 1]])
    }
}
if p.total(chain) ≠ lb {
    log.Fatalf("value␣mismatch:␣%d␣!=␣%d", p.total(chain), lb)
}
```

This code is used in section 32.

34. The chain is printed exactly as FOOTBALL prints one: the date, then each team with its nickname and score, then the running total in parentheses. Because we are maximizing a sum and not a minimum, a game that u lost may well belong to the best chain, so the running total sometimes dips.

⟨ Print the chain game by game 34 ⟩ ≡

```
var run int64
for k := 0; k + 1 < len(chain); k++ {
    u, v := chain[k], chain[k + 1]
    run += p.del[u][v]
    d := p.date[u][v]
    ⟨ Turn day d into a month name mon and a day day 35 ⟩
    fmt.Printf("␣%s␣%02d:␣%s␣%s␣%d,␣%s␣%s␣%d␣(%+d)\n",
        mon, day, p.names[u], p.nick[u], p.uScore[u][v],
        p.names[v], p.nick[v], p.vScore[u][v], run)
}
```

This code is used in section 32.

35. Day 0 was August 26, 1990, and day 128 was January 1, 1991.

⟨ Turn day d into a month name *mon* and a day *day* 35 ⟩ ≡

```
var mon string
var day int64
switch {
case d ≤ 5:
    mon, day = "Aug", d + 26
case d ≤ 35:
    mon, day = "Sep", d − 5
case d ≤ 66:
    mon, day = "Oct", d − 35
case d ≤ 96:
    mon, day = "Nov", d − 66
case d ≤ 127:
    mon, day = "Dec", d − 96
default:
    mon, day = "Jan", 1
}
```

This code is used in section 34.

36. Index.

a: 8, 14, 16.
Add: 22.
After: 22.
AllArcs: 8.
any: 21.
append: 13, 18, 20, 21, 23, 24, 28.
B: 8.
banned: 11, 18, 24.
base: 11, 13, 18.
bbHeap: 21, 22.
bbNode: 11, 18, 21, 22.
bd: 13, 18, 19, 20, 22, 23, 24, 25, 27, 30.
best: 20.
Bool: 4.
bool: 5, 7, 14, 20, 21, 22, 27, 33.
bound: 11, 18, 21, 22, 23, 24, 25.
bounder: 11, 18, 20, 22, 27.
buildProblem: 3, 6.
c: 13, 19, 28, 30.
cc: 19.
chain: 32, 33, 34.
child: 24.
colOf: 11, 13, 18, 19.
cols: 11, 13, 18.
ctot: 30.
cur: 16.
cyc: 20, 23, 24.
cycles: 27, 28, 30.
d: 8, 34, 35.
date: 5, 7, 8, 34.
day: 34, 35.
deadline: 22.
decompose: 27, 30.
del: 5, 6, 7, 8, 9, 13, 20, 27, 28, 34.
delta: 16.
dir: 3, 4, 6.
Duration: 4, 22.
e: 18, 19.
err: 3, 6.
error: 6.
Errorf: 6.
exists: 5, 7, 8, 13, 33.
Fatal: 3, 22.
Fatalf: 33.
flag: 4.
fmt: 6, 22, 30, 32, 34.
forbid: 12, 13, 18, 19.
forced: 11, 18, 24.
from: 3, 4, 6, 8, 32.
g: 6, 8.
Games: 6.
gbgames: 6.
goal: 5, 6, 8, 10, 11, 13, 20, 23, 27.
h: 21, 22, 24, 25.
head: 26, 28, 30.
heap: 22, 24.
hungarian: 14, 18.
I: 8.
i: 7, 8, 14, 15, 21.
i0: 16.
inc: 11, 22, 23, 24, 25.
incPath: 11, 22, 23.
Index: 8.
inf: 14, 15, 16.
Init: 22.
int: 5, 6, 8, 9, 11, 13, 14, 18, 20, 21, 22, 24, 26, 27, 33.
int64: 5, 7, 9, 11, 12, 13, 14, 18, 20, 22, 26, 27, 30, 34, 35.
j: 7, 8, 14, 15, 16, 21, 24.
j0: 15, 16, 17.
j1: 16, 17.
k: 9, 23, 24, 33, 34.
lb: 32, 33.
Len: 8, 21, 22, 25.
len: 9, 13, 14, 18, 20, 21, 24, 26, 28, 30, 33, 34.
Less: 21.
limit: 4, 22, 32.
log: 3, 22, 33.
m: 18, 19.
main: 3.
make: 6, 7, 13, 14, 18, 20, 27, 33.
match: 14, 15, 16, 17.
math: 7, 14, 22.
MaxInt64: 14.
MinInt32: 7, 22.
minv: 14, 15, 16.
mon: 34, 35.
N: 6.
n: 5, 6, 7, 8, 13, 14, 15, 16, 18, 20, 21, 27.
Name: 8.
names: 5, 6, 8, 30, 33, 34.
nd: 22, 23, 24.
negInf: 7.

nick: 5, 6, 8, 34.
nodes: 22, 32.
Now: 22.
old: 21.
p: 3, 6, 7, 8, 9, 11, 13, 18, 20, 22, 23, 27, 28, 30, 32, 33, 34.
Parse: 4.
Partner: 8.
path: 9.
piece: 26, 27, 28.
plen: 27, 30.
Pop: 21, 22.
posCycle: 20, 23.
Printf: 22, 30, 32, 34.
problem: 5, 6, 9, 11, 22.
proven: 22, 25, 32.
ptot: 27, 30.
Push: 21, 24.
r: 13, 18, 19.
res: 14, 18.
root: 22, 30.
rowOf: 11, 13, 18, 19.
rows: 11, 13, 18.
rr: 19.
run: 34.
S: 8.
s: 20, 27, 28.
seen: 20, 27, 28, 33.
show: 4, 32.
skipped: 27, 28, 30.
solve: 18, 22, 24.
solveExactly: 22, 32.
start: 5, 6, 8, 10, 11, 13, 20, 23, 27.
String: 4.
string: 5, 6, 35.
succ: 11, 18, 20, 23, 27, 28, 30.
Swap: 21.
t: 9.
time: 22.
Tip: 8.
to: 3, 4, 6, 8, 32.
top: 25.
tot: 20, 26, 28, 30.
total: 9, 14, 18, 33.
u: 5, 14, 16, 34.
ub: 22, 25, 32.
uScore: 5, 7, 8, 34.
used: 14, 15, 16.
uu: 13, 18.
v: 5, 8, 13, 14, 16, 20, 23, 27, 28, 33, 34.
verdict: 32.
Vertices: 8.
vScore: 5, 7, 8, 34.
vv: 13.
way: 14, 15, 16, 17.
x: 21.
Y: 8.

⟨ Accept *nd* as a chain, or branch on one of its cycles 23 ⟩ Used in section 22
⟨ Allocate the matrices 7 ⟩ Used in section 6
⟨ Check that the chain is genuine 33 ⟩ Used in section 32
⟨ Collect the piece that *s* opens 28 ⟩ Used in section 27
⟨ Determine the final bounds 25 ⟩ Used in section 22
⟨ Dissect the root relaxation 30 ⟩ Used in section 22
⟨ Fill the matrices from the vertices and arcs 8 ⟩ Used in section 6
⟨ Find an augmenting path from row *i* and update the assignment 15 ⟩ Used in section 14
⟨ Forbid everything that competes with the forced edge *e* 19 ⟩ Used in section 18
⟨ Make the child that omits the *k*th edge of *cyc* 24 ⟩ Used in section 23
⟨ Print the chain game by game 34 ⟩ Used in section 32
⟨ Read the command line 4 ⟩ Used in section 3
⟨ Retrace *way*, reassigning columns 17 ⟩ Used in section 15
⟨ Set up the rows, the columns, and the base matrix 13 ⟩ Used in section 22
⟨ Solve the problem and report 32 ⟩ Used in section 3
⟨ Step to the unused column of least reduced cost 16 ⟩ Used in section 15
⟨ Subroutines 6 ⟩ Used in section 3
⟨ Turn day *d* into a month name *mon* and a day *day* 35 ⟩ Used in section 34
⟨ Type declarations 5 ⟩ Used in section 3

CHAIN_BOUND

	Section	Page
Introduction	1	1
The problem	5	3
The assignment relaxation	10	6
The Hungarian algorithm	14	8
Solving one node	18	10
Best-first search	21	12
Anatomy of the root relaxation	26	14
Results	31	16
Index	36	19